

THE 8TH KB FUTURE FINANCE A.I. CHALLENGE

다 됐나요?

은행 앱이 말해주지 않는 실패 원인을, 근거와 함께 판정합니다

팀명 **다 됐나요** · 2인

제8회 KB Future Finance A.I. Challenge · 예선 기술설명서

2026-07-29

한 장 요약

문제 — 은행 앱은 "안 됩니다" 라고만 말하고, 무엇이 막고 있는지는 말하지 않습니다.

해결 — KB 공개문서에서 조건을 자동 추출해 기계가 판정할 수 있는 조건 트리로 만들고, 사용자 상태와 대조해 막힌 항목 + 근거 원문 + 다음 행동을 지목합니다.

KB 공개문서

—[LLM]→

조건 트리

—[규칙엔진]→

무엇에 막혔는가

약관·FAQ

추출

재사용 자산

+ 근거 원문

8건

조건 12개

+ 앱에서 되는지

+ 하나만 풀면 되는지

지금 상태	
문서 8건 → 청크 280개 → 조건 12개	전부 자동 추출 · 검수 통과
근거가 원문에 실재	12 / 12
판정 2,000회 반복 일치율	100%
판정 1회 소요 (중앙값)	0.056 ms

1. 문제 정의

앱은 실패를 알려주지만, 원인은 알려주지 않는다

앱: "신분증이 인식되지 않습니다. 다시 촬영해 주세요."

사용자: 재촬영 반복 → 포기

실제: 비대면 계좌개설 안심차단이 켜져 있음

→ 다시 찍어도, 다른 은행에 가도 개설되지 않음

→ 해제는 영업점에서만 가능

"확인하세요"와 "확인해드렸습니다"의 차이. 검색하면 FAQ가 나옵니다. 그러나 FAQ는 **확인하라고** 말할 뿐, 지금 내 상태가 어떤지는 확인해주지 않습니다.

이 프로젝트는 팀장이 직접 겪은 일에서 시작했습니다 — "**해외결제를 풀려고 하나 풀었는데 안 되고, 하나 더 풀 게 있더라. 어디 있는지 몰라 검색했다.**"

2. 문제의 크기 — 추정이 아니라 실측

조건은 흩어져 있고, 화면은 일부만 보여준다

	값	출처
KB Pay 설정 화면에 보이는 항목	2개	실측 (2026-07-27)
공개문서에서 자동 추출한 조건	11개	<code>data/trees/</code>
그중 앱에서 해결할 수 없는 조건	8개	<code>remedy.actionable_in_app</code>
사람이 조건 7개를 직접 확인한 시간	326초	스톱워치 실측, 측정자 2인
그중 검색을 거친 조건	4 / 7	앱 메뉴로 도달한 사례 0건
결론에 도달하지 못한 조건	1건	FAQ가 "고객센터 문의"로 종결
인증 장벽으로 측정조차 못한 조건	2건	카드 비밀번호 미기억

326초는 **찾아야 할 목록을 미리 알고 켜** 시간입니다. 무엇을 확인해야 하는지 모르는 실제 사용자는 더 오래 걸립니다 — 이 수치는 **하한**입니다. (7개는 측정 시점 기준이며, 자동 추출본은 11개입니다)

3. 접근 — 생성이 아니라 검증

Retrieval-Augmented Verification

일반적인 RAG는 검색 결과를 **문장으로 요약**합니다. 그러면 답이 매번 달라지고, 틀려도 그럴듯합니다.

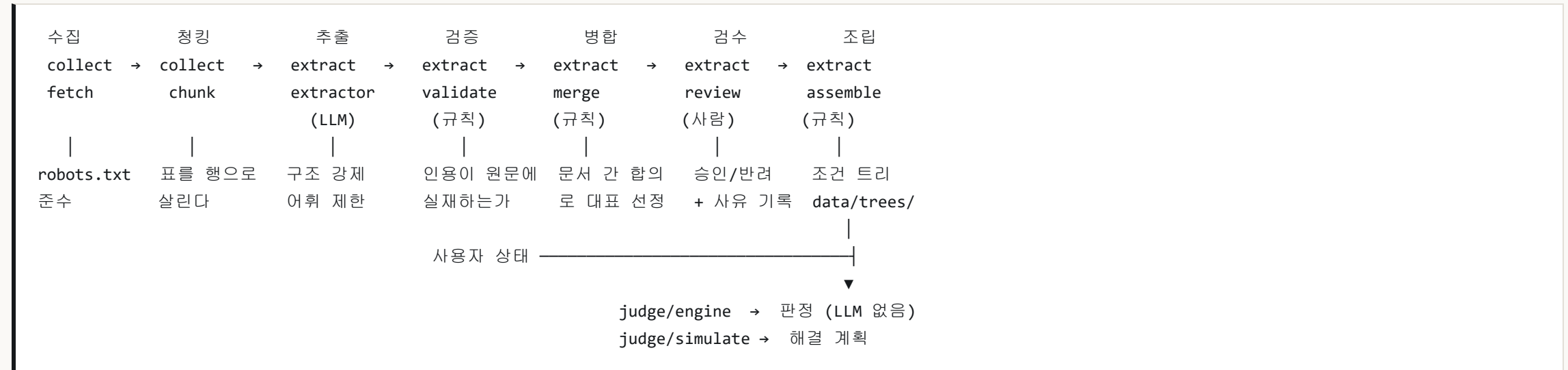
저희는 검색 결과를 **실행 가능한 술어(predicate)**로 바꿉니다.

RAG	문서 → LLM → 자연어 답변	(매번 다름 · 검증 불가)
RAV(우리)	문서 → LLM → 조건 트리(구조)	← 여기까지만 LLM
	↓	
	사용자 상태 → 규칙 엔진 → 판정	← 여기부터는 결정론

	LLM 이 하는 일	LLM 이 하지 않는 일
범위	비정형 문서에서 조건을 뽑아 형식화	판정
결과물	조건 트리 (파일로 고정)	—
재현성	캐시 + <code>temperature=0</code>	같은 입력 = 같은 출력 (측정)

이 분리가 이 제안의 핵심입니다. 금융 판정은 틀리면 사용자를 잘못된 행동으로 보냅니다. 그래서 판정에서 LLM을 뺐고, 뺐다는 사실을 **숫자로 증명**합니다.

4. 시스템 구조



LLM 은 **extract/extractor** 한 곳에만 있습니다. 나머지는 전부 규칙과 사람입니다.

5. 핵심 산출물 — 조건 트리 (C1)

이 프로젝트가 남기는 자산은 서비스가 아니라 데이터입니다

```
{
  "id": "c_nonface_open_block",
  "label": "비대면 계좌개설 안심차단이 해제되어 있어야 합니다",
  "predicate": { "subject": "account.nonface_open_block", "op": "eq", "value": false },
  "severity": "blocking",
  "remedy": { "actionable_in_app": false, "channels": ["branch:영업점"],
    "primary_path": "영업점 방문 (신분증 지참)" },
  "evidence": {
    "source_title": "비대면 계좌개설 안심차단 서비스",
    "url": "https://obank.kbstar.com/quics?page=C112454",
    "quote": "서비스 해제는 영업점에서만 가능합니다.",
    "collected_at": "2026-07-29", "confidence": "high"
  },
  "provenance": { "support_count": 5, "source_ids": ["kbthink_safeblock", "obank_nonface_safeblock"] }
}
```

- `evidence` 는 스키마상 필수입니다 → 근거 없는 조건은 존재할 수 없습니다
- `actionable_in_app` 이 앱에서 못 고치는 조건을 데이터로 구분합니다
- `provenance` 는 몇 개 문서에서 확인됐는지를 남깁니다

6. 추출 — 통제된 어휘로 형식화한다

directed symbolic prompting

LLM에게 자유롭게 조건을 만들게 하면 존재하지 않는 상태 경로를 지어냅니다. 그 조건은 사용자 프로필과 매칭되지 않아 영원히 `unknown` 이 됩니다.

그래서 참조 가능한 어휘를 미리 못 박습니다.

	수	예
허용 <code>subject</code> (상태 경로)	21	<code>card.overseas_block_online</code> , <code>account.nonface_open_block</code>
허용 <code>op</code> (연산자)	16	<code>eq</code> , <code>lte</code> , <code>date_after</code> , <code>count_lte</code>
허용 <code>category</code>	5	setting / document / limit / temporal / eligibility

근거: **Neuro-Symbolic Framework for Public-Sector AI** (arXiv:2512.12109, FAccT 2026) — 통제 어휘를 명시하면 형식화 성공률이 크게 오른다.

목록에 맞는 경로가 없으면 그 조건은 추출하지 않습니다. 빠뜨리는 비용이 틀리는 비용보다 작기 때문입니다.

7. 검증 게이트 — "근거 100%"를 구조로 보증한다

말이 아니라 코드가 막는다

#	게이트	막는 것
1	인용 실재 검증	모델이 지어낸 인용 — 원문에 없으면 폐기
2	<code>subject</code> 허용목록	존재하지 않는 상태 경로
3	<code>op</code> 허용목록	평가할 수 없는 연산자
4	<code>value</code> JSON 파싱	빈 값·형식 오류
5	<code>confidence</code> 규칙	medium/low 인데 사유가 없는 조건

인용 자동 보정

모델은 원문 문장 앞뒤에 **없는 말을 덧붙이는** 실수를 반복했습니다. 그대로 폐기하면 **실재하는 조건이 사라집니다**.

원문	"... 권유 드립니다. 원화 결제 시 추가로 부과되는 ..."
모델	"해외 가맹점에서 원화 결제 시 추가로 부과되는 ..." ← 앞에 없는 구절
보정	원문에 실제로 있는 최장 연속 구간만 남긴다 → 결과는 언제나 원문의 부분 문자열

LLM 에게 사유를 알려주고 다시 묻는 방법을 **먼저 시도했습니다** — **3건 중 0건 복구**. 같은 실수를 반복했습니다. 그래서 규칙으로 바꿨고, 1건을 되살렸습니다.

8. 프롬프트 규칙 — 추측이 아니라 관측

1차 추출을 분석해 실패 7종을 규칙으로 못 박았다

#	관측된 실패	규칙
①	방향 뒤집힘 — "안심차단이 켜져 있어야 한다"	이 predicate 가 참이면 목표가 달성되는가?
②	subject 오매핑 — 서로 다른 조건을 한 경로에 몰아넣음	뜻이 정확히 맞지 않으면 추출하지 않는다
③	목표와 무관 — 이의신청 기한, 수수료 면제	지금 그 목표를 달성하는 조건만
④	단위 혼동 — "5,500달러" → 5500000	원화 기준이 아니면 추출하지 않는다
⑤	인용 이어붙임	연속된 한 구간만
⑥	<code>value</code> 를 비움	값이 없어도 <code>"null"</code> 을 적는다
⑦	label 과 predicate 방향 불일치	label 을 predicate 로 다시 쓰면 같은 문장인가?

1차 결과는 병합 15개 중 8개가 위 유형으로 틀렸습니다. 규칙을 넣고 5차까지 개정한 결과가 지금의 트리입니다.

이 규칙들은 실제로 나온 오류입니다. 지을 때는 그 실패가 재현되는지 먼저 확인하도록 코드 주석에 남겼습니다.

9. 병합 — 문서 간 합의로 대표를 정한다

길이는 신뢰의 근거가 아니다

같은 조건이 여러 문서에 조금씩 다른 문장으로 실립니다. 무엇을 보여줄지 정해야 합니다.

- X 짧은 인용 우선 → 규칙을 스치듯 언급한 문장이 대표가 됨
- X 긴 인용 우선 → 다른 얘기를 하는 긴 문단이 대표가 됨
("해외거래정지 해제"(문서 5곳)가 "해외사이트 해제신청"(1곳)에 밀림)
- o 문서 간 합의 → 같은 뜻으로 읽은 문서가 가장 많은 무리를 대표로
그 안에서 (여러 문서에 같은 문장 → label 뒷받침 → 짧은 것) 순

합집합을 쓰지 않는 이유

해결 채널을 문서들의 합집합으로 모았더니, "앱에서 안심차단을 신청할 수 있어요"의 app:KB스타뱅킹 이 해제 조건에 섞여 "앱에서 해제 가능"으로 뒤집혔습니다.

→ 화면에 보이는 모든 정보는 하나의 인용에서 나옵니다. 문서마다 값이 다르면 합치지 않고 검수 메모로 사람에게 넘깁니다.

10. 검수 관문 — 사람은 게이트지 저자가 아니다

프롬프트를 고쳐도 오류가 0이 되지는 않습니다. 틀린 조건이 트리에 들어가면 판정은 자신 있게 틀린 답을 냅니다. 그게 가장 나쁩니다.

사람이 할 수 있는 것	사람이 할 수 없는 것
승인 / 반려	<code>label</code> 을 쓰거나 고치기
<code>severity</code> 교정 (blocking ↔ warning)	<code>predicate</code> 수정
여러 인용 중 대표 선택 (모델이 뽑은 것 중에서만)	<code>evidence</code> 문장 작성

검수 대상 16건 → 승인 12 / 반려 4 (통과율 0.75)

`severity` 교정 2 · 대표 인용 선택 1

- 결정과 사유 전문이 `data/review/decisions.yaml` 에 남습니다 — 감사 기록입니다
- 교정한 조건은 트리에 `review_override` 로 표시돼 손댄 곳이 감춰지지 않습니다
- 검수되지 않은 조건은 트리에 들어가지 않습니다 (모르면 넣지 않는다)

11. 판정 엔진 — 결정론

같은 입력에 같은 출력. 측정했습니다.

항목	값
조합 (트리 × 프로필)	10
총 판정 횟수	2,000
일치율	100.0%
판정 1회 소요 (중앙값)	0.056 ms

비교 시 실행 시각을 제외하고 조건 목록을 `id` 로 정렬해 대조했습니다. 판정과 해결 계획을 **함께** 직렬화해 비교했으므로, 둘 중 하나만 흔들려도 불일치로 잡힙니다.

조합에는 **목표와 무관한 프로필**(계좌 트리 × 해외결제 사용자)도 넣었습니다. 이 경우 시스템은 `indeterminate` 를 냅니다 — 관련 정보가 없으면 추측하지 않습니다.

11-1. 판정은 3값입니다

값	뜻
<code>blocked</code>	차단 조건이 미충족 — 지금은 되지 않습니다
<code>indeterminate</code>	미충족은 없지만 판정에 필요한 값을 몰라 된다고 말할 수 없습니다
<code>ok</code>	차단 조건을 전부 확인했고 모두 충족

`indeterminate` 가 있는 이유 — **모르는 상태에서 "됩니다"라고 답하지 않기 위해서**입니다. 값을 모르는데 `ok` 를 주면 그건 추측이고, 이 시스템의 설계 원칙 위반입니다.

전부 $\checkmark/\times$ 로 채운 화면이 더 깔끔해 보이지만, **모르는 것을 안다고 표시하는 것이 거짓말**입니다. 그래서 `unknown` 을 화면에 그대로 노출합니다.

12. 해결 계획 — "하나만 풀면 되나요?"

조건을 나열해 주는 것만으로는 이 프로젝트가 시작된 경험이 해결되지 않습니다. **하나를 풀어도 여전히 막힌다는 사실을** 먼저 말해야 합니다.

판정 - 조건 11개 중 미충족 3 · 확인불가 1

- × 해외원화결제(DCC) 차단 해제 앱에서 가능: KB Pay
- × 해외거래정지 해제 앱에서 가능: 같은 화면 별도 항목
- × IC칩 비밀번호 등록 앱에서 불가: 영업점 또는 ATM

— 하나만 풀면 되나요? —————

- DCC 차단만 해제 → 여전히 안 됩니다 · 필수 조건 2개 남음
- 해외거래정지만 해제 → 여전히 안 됩니다 · 필수 조건 2개 남음
- 앱에서 할 수 있는 것 모두 → 여전히 안 됩니다 · 필수 조건 1개 남음
- 3개 모두 → 됩니다

이 계산에도 LLM을 쓰지 않습니다. 판정 결과를 집합 연산으로 다시 평가할 뿐이며, **없는 사용자 값을 지어내지 않습니다** — 값을 만들면 그 값이 맞는지 아무도 보증할 수 없습니다.

13. 측정 결과

지표	방법	결과
조건 추출 규모	문서 8건 → 청크 280 → 후보 276 → 병합 16	조건 12개
검수 통과율	병합된 조건 중 트리에 들어간 비율	12 / 16 = 0.75
근거 실재율	트리의 모든 인용이 원문에 있는가	12 / 12 = 100%
근거 표시율	판정 결과에 근거가 붙는 비율	구조적 100% (스키마 필수)
앱에서 해결 불가	<code>actionable_in_app == false</code>	해외결제 8 / 11
결정론성	2,000회 반복 일치율	100%
수동 라벨 대비	정밀도 / 재현율	<code>eval/results/extraction_accuracy.md</code>

13-1. 수동 라벨과의 차이를 정직하게 다룹니다

- 자동 추출이 **라벨에 없던 유효 조건 4개**를 새로 찾았습니다

(IC칩 비밀번호 · 명의도용 차단 · 카드 잠금 · 1일 한도)

- 반대로 라벨에만 있던 1건은 **검토 결과 라벨 쪽이 틀렸습니다** — 방향을 뒤집어 만든 조건이었고,

LLM 이 조건이 아니라고 판정한 것이 옳았습니다

- **라벨 파일은 고치지 않았습니다.** 자를 나중에 고치면 그 자로 켜 수치가 의미를 잃습니다.

결함은 `eval/labels/errata.yaml` 에 기록하고 보고서가 함께 출력합니다

14. 한계 — 숨기지 않습니다

한계	우리의 처리
사용자 상태는 가상입니다	실제 연동에는 내부 API가 필요합니다. 이 시스템에서 가짜는 상태 하나뿐이고, 조건·근거는 전부 실제 문서에서 나옵니다
판정 응답 0.056ms 는 로직 비용 입니다	실제 서비스에는 상태 조회 시간이 더해집니다. 그래서 속도 배수를 강조하지 않습니다
약관은 개정됩니다	모든 조건에 수집 일자 를 남기고, 트리의 유효성은 가장 낡은 근거를 따릅니다
표에서 뽑은 인용 1건	셀을 `w` `로 이은 형태라 원문 HTML 검색으로는 나오지 않습니다 (각 셀은 실재). README에 명시
커버리지	지금은 목표 2종입니다. 조건 트리는 목표 단위로 추가되는 구조 입니다
수집 범위	<code>robots.txt</code> 가 막은 페이지는 자동 수집하지 않고, 사람이 공개 안내 본문만 확보해 <code>collection_method: manual</code> 로 기록합니다

로그인 뒤 개인화 페이지는 수집하지 않습니다. 공시·안내 페이지만 사용합니다.

15. 개발 계획

예선 이후 — 무엇을 어떤 순서로

단계	내용	검증 방법
1. 목표 식별	자연어 질의 → goal_id (<code>src/resolve/</code>)	질의 100건 라벨 대비 정확도. 못 찾으면 "아직 다루지 않는 주제" 로 정직하게 답함
2. 커버리지 확장	목표 2종 → 10종 (대출·카드발급·이체한도 등)	목표당 조건 추출 → 검수 통과율 유지 여부
3. 상태 연동	가상 프로필 → 내부 API	조회 실패 시 <code>unknown</code> 유지 (추측 금지)
4. 개정 감시	문서 재수집 → 조건 변경 감지	트리 diff 알림. 변경된 조건은 재검수 대상
5. 운영 지표	판정 후 실제 해결 여부 추적	"지목한 원인이 맞았는가" 사후 검증

확장이 쉬운 이유

새 목표를 넣을 때 코드를 고치지 않습니다. `sources.yaml1` 에 문서를 추가하고 → 파이프라인을 돌리고 → 검수하면 트리가 생깁니다.

16. 기술 실현 가능성

지금 돌아갑니다. 제3자도 돌릴 수 있습니다.

코드 규모	파이썬 27개 파일 / 2,604줄 (최대 파일 196줄)
외부 의존	<code>requests</code> · <code>PyYAML</code> · <code>pydantic</code> · <code>fastapi</code> · <code>openai</code> — HTML 파서도 표준 라이브러리
실행	<code>pip install -r requirements.txt</code> → <code>uvicorn src.api.main:app</code>
재현 절차	<code>docs/REPRODUCE.md</code> — 새 폴더에서 처음부터 돌린 로그
LLM 비용	조건 추출은 오프라인 1회성 . 판정에는 호출이 없습니다

유지보수를 전제로 만들었습니다

- **파일 200줄 / 함수 40줄** 상한 (전 파일 준수)
- 파이프라인 단계 = 폴더 하나. 새 단계가 생기면 폴더를 만들지, 기존 파일을 부풀리지 않습니다
- 인터페이스 계약 C1~C4를 `judge/schema.py` 한 곳에 고정 — 세 사람이 병렬로 일하는 접점입니다

보안

`.env` 는 `.gitignore` + **pre-commit** **훅**으로 이중 차단하며, 훅은 감지해도 **값을 출력하지 않습니다**. LLM 캐시에는 응답 구조체만 저장하고 요청 헤더는 남기지 않습니다.

17. 팀과 역할

담당

팀장	문제 정의 · 조건 추출 설계(프롬프트·어휘·검수) · 근거 검증
팀원	검증 스위트(결정론성·엣지) · 클린설치 재현 · 기술 문서

AI 활용

코드 작성·문서 정리에 **Claude** 를 활용했습니다. 다만 무엇을 만들지, 어떤 조건을 트리에 넣을지, 어떤 수치를 주장할지는 사람이 판단했습니다.

이 문서의 모든 수치는 **저장소의 코드와 데이터에서 직접 산출**한 값입니다. 추정치는 쓰지 않았고, 측정하지 못한 것은 측정하지 못했다고 적었습니다.

다 됐나요?

조치를 하나 끝내고 재시도하며 던지는 질문이자, 앱이 끝내 대답하지 않는 질문입니다.

역대 팀들은 약관을 사람에게 쉽게 설명했습니다. 저희는 약관을 기계가 판정할 수 있게 만듭니다.

저장소 · 데모 · 재현 절차는 제출물에 포함되어 있습니다.

모든 조건에는 출처 URL · 원문 인용 · 수집 일자가 붙어 있습니다.